



Department of Electrical and Computer Engineering

Summer Semester, 2024/2025

ENCS3310 Advanced Digital Systems Design

Design and Verification of a Simplified Memory Controller

Prepared by:

Lana Ayed Sayes

1220785

Supervised by:

Dr. Ayman Hroub

Table of Contents:

Table of Contents:	II
Table of Figures	III
1. Introduction :	1
2. Block diagram of the system:	1
3. Memory controller RTL code:	2
● Controller code :	2
4. Memory controller TestBecnh code:	8
5. Memory controller simulation screenshots:	12
6. Complete system RTL code:	12
● system RTL code:	12
7. Complete system TestBecnh code:	25
● system TestBecnh	25
8. Complete system simulation screenshots:	30
9. Conclusion	30

Table of Figures

Figure 1 System	1
Figure 2 Block diagram of the system	1
Figure 3 state diagram	2
Figure 5 Memory controller simulation	12
Figure 6 Complete system simulation	30

1. Introduction :

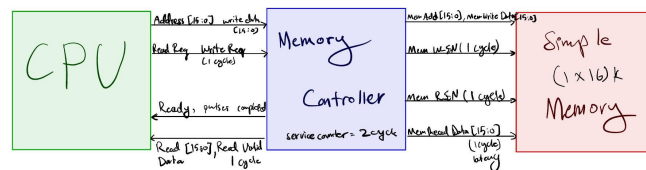
The project offers a small memory-controller subsystem that connects a CPU block with a synchronous 1K×16 RAM and meets the course requirements. The design utilizes a simple control procedure with a small finite-state machine that accepts a request, generates a one-cycle memory strobe, adds a fixed two-cycle service time, and then signals completion by a one-cycle pulse. When read and write requests arrive in the same clock, the controller writes ahead of time before providing an immediate read, ensuring deterministic order. A self-starting active-low reset resets all the blocks to a known state. A self-testing testbench integrates the CPU driver, controller, and memory and verifies write→readback correctness, one-cycle pulse widths, the two-cycle latency, and the concurrent read-write case. Waveform evidence verifies the anticipated timing and handshake behavior.



Figure 1 System

2. Block diagram of the system:

The diagram illustrates the system at an elevated level: the CPU requests reads/writes (address and data) from the memory controller, which schedules them through a tiny FSM. The controller asserts one-cycle strobes to the synchronous 1K×16 RAM, maintains address/data holding, imposes a fixed two-cycle service time, and provides one-cycle completion pulses to the CPU. If read and write both arrive in the same clock, the controller writes first and schedules the read to run immediately after that. The ready signal is asserted in IDLE (without a pending read) to indicate that the controller can accept a new request.



Memory controller → handle Pending Read (addr latch), Strobe (Idle, write, read)

Figure 2 Block diagram of the system

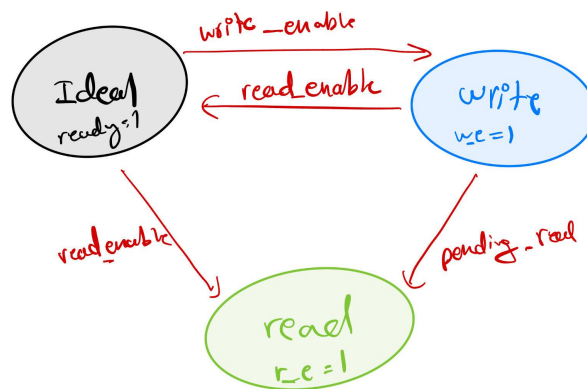


Figure 3 state diagram

Since WRITE is the priority, set pending_read to serve READ as soon as WRITE is finished if write_enable and read_enable are set together.

Moore FSM: outputs are state-dependent (ready in IDLE, w_e in WRITE, r_e in READ); IDLE is forced by RST_n=0; write_done/read_valid pulses fire upon operation completion.

3. Memory controller RTL code:

The controller is a small FSM (IDLE, WRITE, READ) with an asynchronous active-low reset, one-cycle strobes toward memory, and one-cycle completion pulses back to the CPU. Each request is served with a fixed two-cycle latency enforced by a simple counter. If read and write arrive in the same clock, the design gives write priority and queues the read to run immediately afterward using a small pending-read latch. The code stabilizes address/data during service and asserts ready only in IDLE when no read is pending

● Controller code :

//Lana Ayed Sayes 1220785

```

module memory_controller (//memory_controller
    input wire    clk,
    input wire    rst_n, //async, active-low reset(on when it's value=0
    input wire [15:0] cpu_addr, //CPU interface
    input wire [15:0] cpu_wdata,
    output reg [15:0] cpu_rdata,
    input wire    cpu_r_en, //1-cycle read request from CPU
    input wire    cpu_w_en, //1-cycle write request from CPU

```

```

output wire    ready,//to announce that a new request can enter
output reg     read_valid,//1-cycle pulse: read data now valid on cpu_rdata
output reg     write_done,//1-cycle pulse: write completed

output reg [15:0] mem_addr, //memory interface,address driven to memory

output reg [15:0] mem_wdata,//write data driven to memory

input wire [15:0] mem_rdata, //read data coming from memory 1-cycle late
output reg     mem_r_en,//1-cycle strobe to start memory read
output reg     mem_w_en//1-cycle strobe to start memory read
);
localparam S_IDLE = 2'd0; //states and control
localparam S_WRITE = 2'd1;
localparam S_READ = 2'd2;

reg [1:0] state, state_n;
reg [15:0] cnt, cnt_n; //fixed total service = 2 cycles

reg pend_read_valid, pend_read_valid_n; //pending READ captured when read+write arrive together,write
has priority

reg [15:0] pend_read_addr, pend_read_addr_n;

//next value holders for registered outputs
reg [15:0] mem_addr_n;
reg [15:0] mem_wdata_n;
reg [15:0] cpu_rdata_n;
reg mem_r_en_n, mem_w_en_n;
reg read_valid_n, write_done_n;

//ready when idle and no queued read
assign ready = (state == S_IDLE) && !pend_read_valid;

```

```

//combinational next-state
always @* begin

    state_n      = state;    //defaults values hold
    cnt_n        = cnt;

    mem_addr_n    = mem_addr;
    mem_wdata_n   = mem_wdata;
    cpu_rdata_n   = cpu_rdata;

    mem_r_en_n    = 1'b0;
    mem_w_en_n    = 1'b0;
    read_valid_n   = 1'b0;
    write_done_n   = 1'b0;

    pend_read_valid_n = pend_read_valid;
    pend_read_addr_n = pend_read_addr;

    case (state)

        S_IDLE: begin//IDLE state case handling
            cnt_n = 16'd0;

            if (cpu_w_en && cpu_r_en) begin
                //handling the condition both read write in the same time simultaneous requests → WRITE first, queue
                READ

                pend_read_valid_n = 1'b1;
                pend_read_addr_n = cpu_addr;

                mem_addr_n      = cpu_addr;
                mem_wdata_n     = cpu_wdata;

```

```

    mem_w_en_n    = 1'b1;    //1-cycle pulse to RAM

    state_n       = S_WRITE;

    cnt_n         = 16'd1;    //fixed 2-cycle service total
end

else if (cpu_w_en) begin//write
    mem_addr_n    = cpu_addr;
    mem_wdata_n   = cpu_wdata;
    mem_w_en_n    = 1'b1;

    state_n       = S_WRITE;

    cnt_n         = 16'd1;
end

else if (cpu_r_en) begin
    mem_addr_n    = cpu_addr;
    mem_r_en_n    = 1'b1;

    state_n       = S_READ;

    cnt_n         = 16'd1;
end

end

end

S_WRITE: begin //WRITE case handling
    if (cnt == 16'd0) begin
        write_done_n = 1'b1;//complete write then pulse write_done
        if (pend_read_valid) begin
            //start queued READ immediately
            mem_addr_n    = pend_read_addr;
            pend_read_valid_n = 1'b0;
            mem_r_en_n    = 1'b1;

            state_n       = S_READ;

```



```

        cnt_n      = 16'd1;//start next 2 cycle service
    end else begin
        state_n = S_IDLE;
    end
end else begin
    cnt_n = cnt - 16'd1;
end
end

//READ
S_READ: begin
    if (cnt == 16'd0) begin
        cpu_rdata_n = mem_rdata;//drive read data back to CPU
        read_valid_n = 1'b1;      //data valid now 1 cycle completion pulse
        state_n     = S_IDLE;//return to idle after completing
    end
else
    begin
        cnt_n = cnt - 16'd1;//count down
    end
end

default: state_n = S_IDLE;
endcase
end

always @(posedge clk or negedge rst_n) begin //sequential (async reset)

    if (!rst_n) begin
        state     <= S_IDLE;//start from idle
    end
end

```

```

cnt          <= 16'd0;//no serv in progress

pend_read_valid <= 1'b0;//clear any queued READ
pend_read_addr <= 16'h0000;//address dont care on reset

mem_addr      <= 16'h0000;//clear data path regs
mem_wdata     <= 16'h0000;
cpu_rdata     <= 16'h0000;

mem_r_en      <= 1'b0;//all flags set to zero
mem_w_en      <= 1'b0;
read_valid    <= 1'b0;
write_done    <= 1'b0;
end else begin
    state      <= state_n;
    cnt        <= cnt_n;

    pend_read_valid <= pend_read_valid_n;//update pending read flag
    pend_read_addr <= pend_read_addr_n;

    mem_addr      <= mem_addr_n;//drive memory address
    mem_wdata     <= mem_wdata_n;
    cpu_rdata     <= cpu_rdata_n;

    mem_r_en      <= mem_r_en_n;//1-cycle read  registered
    mem_w_en      <= mem_w_en_n;
    read_valid    <= read_valid_n;//1-cycle pulse to signal read complete
    write_done    <= write_done_n;//1-cycle pulse to signal WRITE complete
end
end
endmodule

```

4. Memory controller TestBench code:

```
module tb_top;

    reg clk;    //simulation clock
    reg rst_n;  //async, active-low reset

    initial clk = 1'b0;    //start clock low
    always #5 clk = ~clk;  //100 MHz clock (period = 10 ns)

    initial begin
        rst_n = 1'b0;    //hold reset active (low)
        #25;             //keep reset for 25 ns
        rst_n = 1'b1;    //release reset (start normal operation)
    end

    wire [15:0] cpu_addr;    //CPU request address
    wire [15:0] cpu_wdata;   //CPU write data
    wire [15:0] cpu_rdata;   //data returned to CPU on reads
    wire    cpu_r_en;        //1-cycle CPU read request
    wire    cpu_w_en;        //1-cycle CPU write request
    wire    ready;           //controller ready to accept a new request
    wire    read_valid;      //1-cycle: cpu_rdata is valid now
    wire    write_done;      //1-cycle: write transaction completed

    wire [15:0] mem_addr;    // address driven to memory
    wire [15:0] mem_wdata;   // write data driven to memory
    wire [15:0] mem_rdata;   //data read back from memory
    wire    mem_r_en;        //1-cycle read strobe to memory
```

```
wire    mem_w_en;    //1-cycle write strobe to memory
```

```
memory_controller u_ctrl (  
    .clk    (clk),  
    .rst_n  (rst_n),    // async reset, active-low  
  
    //CPU side (requests/responses)  
    .cpu_addr (cpu_addr),  
    .cpu_wdata (cpu_wdata),  
    .cpu_rdata (cpu_rdata),  
    .cpu_r_en (cpu_r_en),  
    .cpu_w_en (cpu_w_en),  
    .ready   (ready),  
    .read_valid(read_valid),  
    .write_done(write_done),  
  
    //memory side (to RAM model)  
    .mem_addr (mem_addr),  
    .mem_wdata (mem_wdata),  
    .mem_rdata (mem_rdata),  
    .mem_r_en (mem_r_en),  
    .mem_w_en (mem_w_en)  
);
```

```
simple_memory u_mem (  
    .clk    (clk),    //same clock domain  
    .rst_n  (rst_n),    //async reset  
    .mem_addr (mem_addr),    //address from controller  
    .mem_wdata(mem_wdata),    //write data from controller  
    .mem_rdata(mem_rdata),    //read data back to controller  
    .mem_r_en (mem_r_en),    //read strobe
```

```

        .mem_w_en(mem_w_en)    //write strobe
    );

    reg saw_rw_same_cycle;      //flag: saw same-cycle RW when ready=1
    reg expect_wd_first;        //waiting to see write_done first
    reg expect_rv_next;         //after write_done, allow read_valid

    reg prev_write_done;        //last-cycle value for write_done
    reg prev_read_valid;        //last-cycle value for read_valid

    //checker logic on each clock (with async reset)
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            //clear checker state on reset
            saw_rw_same_cycle <= 1'b0;
            expect_wd_first  <= 1'b0;
            expect_rv_next   <= 1'b0;
            prev_write_done  <= 1'b0;
            prev_read_valid  <= 1'b0;
        end else begin
            //detect same-cycle READ+WRITE request when controller is ready
            if (ready && cpu_r_en && cpu_w_en) begin
                saw_rw_same_cycle <= 1'b1;    //arm ordering check
                expect_wd_first  <= 1'b1;    //require write_done first
                expect_rv_next   <= 1'b0;    //then accept read_valid
            end

            //enforce ordering: write_done before read_valid
            if (saw_rw_same_cycle) begin
                if (expect_wd_first && read_valid) begin

```

```

    $display("[TB] ERROR: read_valid asserted before write_done after same-cycle RW (t=%0t)",
$time);

    $fatal;          // fail immediately
end

if (expect_wd_first && write_done) begin
    expect_wd_first <= 1'b0; //write_done seen
    expect_rv_next <= 1'b1; //now wait for read_valid
end

if (expect_rv_next && read_valid) begin
    //correct order observed: clear flags
    expect_rv_next <= 1'b0;
    saw_rw_same_cycle <= 1'b0;
end

end

//pulse-width checks: must be one cycle only
if (prev_write_done && write_done) begin
    $display("[TB] ERROR: write_done wider than 1 cycle at t=%0t", $time);
    $fatal;
end

if (prev_read_valid && read_valid) begin
    $display("[TB] ERROR: read_valid wider than 1 cycle at t=%0t", $time);
    $fatal;
end

//remember current values for next-cycle pulse check
prev_write_done <= write_done;
prev_read_valid <= read_valid;

end

end

```

```
// end simulation after a timeout

initial begin

    #1000;           // run for 1000 ns

    $display("[TB] Simulation finished at t=%0t", $time);

    $finish;         // stop simulation cleanly

end
```

endmodule

5. Memory controller simulation screenshots:

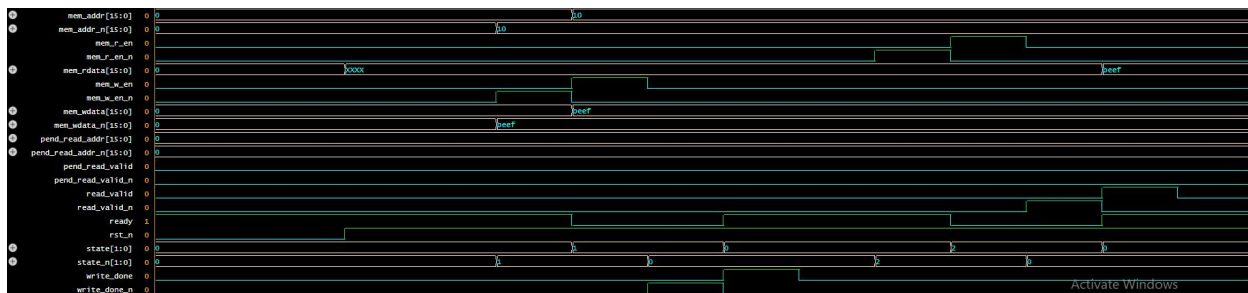


Figure 5 Memory controller simulation

6. Complete system RTL code:

- system RTL code:

```
`timescale 1ns/1ps

//Lana Ayed Sayes 1220785

module memory_controller (//memory_controller

    input wire    clk,

    input wire    rst_n, //async, active-low reset(on when it's value=0
```

```

input wire [15:0] cpu_addr, //CPU interface
input wire [15:0] cpu_wdata,
output reg [15:0] cpu_rdata,
input wire      cpu_r_en, //1-cycle read request from CPU
input wire      cpu_w_en, //1-cycle write request from CPU
output wire      ready, //to announce that a new request can enter
output reg       read_valid, //1-cycle pulse: read data now valid on cpu_rdata
output reg       write_done, //1-cycle pulse: write completed

output reg [15:0] mem_addr, //memory interface, address driven to memory

output reg [15:0] mem_wdata, //write data driven to memory

input wire [15:0] mem_rdata, //read data coming from memory 1-cycle late
output reg       mem_r_en, //1-cycle strobe to start memory read
output reg       mem_w_en, //1-cycle strobe to start memory read
);

localparam S_IDLE = 2'd0; //states and control
localparam S_WRITE = 2'd1;
localparam S_READ = 2'd2;

reg [1:0] state, state_n;
reg [15:0] cnt, cnt_n; //fixed total service = 2 cycles

reg pend_read_valid, pend_read_valid_n; //pending READ captured when read+write arrive together, write
has priority

reg [15:0] pend_read_addr, pend_read_addr_n;

```



```

//next value holders for registered outputs
reg [15:0] mem_addr_n;
reg [15:0] mem_wdata_n;
reg [15:0] cpu_rdata_n;
reg      mem_r_en_n, mem_w_en_n;
reg      read_valid_n, write_done_n;

//ready when idle and no queued read
assign ready = (state == S_IDLE) && !pend_read_valid;

//combinational next-state
always @* begin

    state_n      = state;    //defaults values hold
    cnt_n        = cnt;

    mem_addr_n    = mem_addr;
    mem_wdata_n   = mem_wdata;
    cpu_rdata_n   = cpu_rdata;

    mem_r_en_n    = 1'b0;
    mem_w_en_n    = 1'b0;
    read_valid_n  = 1'b0;
    write_done_n  = 1'b0;

    pend_read_valid_n = pend_read_valid;
    pend_read_addr_n = pend_read_addr;

    case (state)

        S_IDLE: begin//IDLE state case handling
            cnt_n = 16'd0;

```

```

if (cpu_w_en && cpu_r_en) begin
    //handling the condition both read write in the same time simultaneous requests → WRITE first, queue
    READ

    pend_read_valid_n = 1'b1;
    pend_read_addr_n = cpu_addr;

    mem_addr_n      = cpu_addr;
    mem_wdata_n     = cpu_wdata;
    mem_w_en_n      = 1'b1;    //1-cycle pulse to RAM

    state_n         = S_WRITE;
    cnt_n           = 16'd1;    //fixed 2-cycle service total
end

else if (cpu_w_en) begin//write

    mem_addr_n      = cpu_addr;
    mem_wdata_n     = cpu_wdata;
    mem_w_en_n      = 1'b1;

    state_n         = S_WRITE;
    cnt_n           = 16'd1;
end

else if (cpu_r_en) begin

    mem_addr_n      = cpu_addr;
    mem_r_en_n      = 1'b1;

    state_n         = S_READ;
    cnt_n           = 16'd1;
end

```

```

    end
end

S_WRITE: begin //WRITE case handling
    if (cnt == 16'd0) begin
        write_done_n = 1'b1; //complete write then pulse write_done
        if (pend_read_valid) begin
            //start queued READ immediately
            mem_addr_n    = pend_read_addr;
            pend_read_valid_n = 1'b0;
            mem_r_en_n    = 1'b1;

            state_n      = S_READ;
            cnt_n        = 16'd1; //start next 2 cycle service
        end else begin
            state_n = S_IDLE;
        end
    end else begin
        cnt_n = cnt - 16'd1;
    end
end

//READ
S_READ: begin
    if (cnt == 16'd0) begin
        cpu_rdata_n = mem_rdata; //drive read data back to CPU
        read_valid_n = 1'b1;      //data valid now 1 cycle completion pulse
        state_n     = S_IDLE; //return to idle after completing
    end
    else

```

```

        begin
            cnt_n = cnt - 16'd1;//count down
        end
    end

    default: state_n = S_IDLE;
endcase
end

always @(posedge clk or negedge rst_n) begin //sequential (async reset)

    if (!rst_n) begin
        state      <= S_IDLE;//start from ideal
        cnt        <= 16'd0;//no serv in progress

        pend_read_valid <= 1'b0;//clear any queued READ
        pend_read_addr  <= 16'h0000;//address dont care on reset

        mem_addr      <= 16'h0000;//clear data path regs
        mem_wdata      <= 16'h0000;
        cpu_rdata      <= 16'h0000;

        mem_r_en       <= 1'b0;//all flags set to zero
        mem_w_en       <= 1'b0;
        read_valid     <= 1'b0;
        write_done     <= 1'b0;
    end else begin
        state <= state_n;
        cnt  <= cnt_n;

        pend_read_valid <= pend_read_valid_n;//update pending read flag
    end
end

```

```

    pend_read_addr <= pend_read_addr_n;

    mem_addr      <= mem_addr_n;//drive memory address
    mem_wdata     <= mem_wdata_n;
    cpu_rdata     <= cpu_rdata_n;

    mem_r_en      <= mem_r_en_n;//1-cycle read  registered
    mem_w_en      <= mem_w_en_n;
    read_valid    <= read_valid_n;//1-cycle pulse to signal read complete
    write_done    <= write_done_n;//1-cycle pulse to signal WRITE complete
end
end
endmodule

```

```

/////////////////////////////////////////////////////////////////

```

```

//simple_memory

```

```

`timescale 1ns/1ps

```

```

module simple_memory (

```

```

    input wire    clk,

```

```

    input wire    rst_n,

```

```

    input wire [15:0] mem_addr,

```

```

    input wire [15:0] mem_wdata,

```

```

    output reg [15:0] mem_rdata,

```

```

    input wire    mem_r_en,

```

```

    input wire    mem_w_en

```

```

);

```

```

    //fixed depth (1K). Index with lower 10 bits.

```

```

    reg [15:0] mem [0:1023]; //choosing memory size 2^10 bits

```

```

wire [9:0] addr_idx = mem_addr[9:0];

reg [9:0] rd_addr_q; //one-cycle read address pipeline (READ_LAT = 1)

always @(posedge clk or negedge rst_n) begin //sequential logic with async reset

    if (!rst_n) begin
        rd_addr_q <= 10'd0;
        mem_rdata <= 16'h0000;
    end else begin
        //write on mem_w_en
        if (mem_w_en) begin
            mem[addr_idx] <= mem_wdata;
        end

        if (mem_r_en) begin //capture read address on mem_r_en

            rd_addr_q <= addr_idx;
        end

        mem_rdata <= mem[rd_addr_q]; //present data after 1-cycle latency

    end
end

endmodule

/////////////////////////////////////////////////////////////////

//simple_cpu
`timescale 1ns/1ps

```

```

module simple_cpu (
    input wire    clk,
    input wire    rst_n,

    // To controller
    output reg [15:0] cpu_addr,
    output reg [15:0] cpu_wdata,
    output reg    cpu_r_en,
    output reg    cpu_w_en,

    // From controller
    input wire [15:0] cpu_rdata,
    input wire    ready,
    input wire    read_valid,
    input wire    write_done
);

//states
localparam C_RESET      = 4'd0;
localparam C_WAIT_READY = 4'd1;

localparam C_WRITE_A_PRE = 4'd2;
localparam C_WRITE_A     = 4'd3;
localparam C_WRITE_A_WAIT = 4'd4;

localparam C_READ_A_PRE  = 4'd5;
localparam C_READ_A      = 4'd6;
localparam C_READ_A_WAIT = 4'd7;
localparam C_READ_A_CHECK = 4'd8; // check on rdata_q

localparam C_RW_PRE      = 4'd9;
localparam C_RW_SAME_CYCLE = 4'd10;
localparam C_RW_WAIT_W   = 4'd11;

```

```

localparam C_RW_WAIT_R    = 4'd12;
localparam C_RW_CHECK     = 4'd13; // check on rdata_q

localparam C_DONE         = 4'd14;

reg [3:0] cstate, cstate_n;

// persistent outputs (next values)
reg [15:0] cpu_addr_n, cpu_wdata_n;

//expectations
reg [15:0] expect1; // BEEF
reg [15:0] expect2; // 1234

//1-cycle register of read data & valid
reg [15:0] rdata_q;
reg      read_valid_q;

//next-state
always @* begin
    //defaults

    cstate_n = cstate;
    cpu_addr_n = cpu_addr;
    cpu_wdata_n = cpu_wdata;
    cpu_r_en = 1'b0;
    cpu_w_en = 1'b0;

    case (cstate)
        C_RESET:    cstate_n = C_WAIT_READY;

        C_WAIT_READY: if (ready) cstate_n = C_WRITE_A_PRE;
    endcase
end

```



```

//WRITE 0xBEEF @0x0010
C_WRITE_A_PRE: begin
    if (ready) begin
        cpu_addr_n = 16'h0010;
        cpu_wdata_n = 16'hBEEF;
        cstate_n = C_WRITE_A;    // pulse next cycle
    end
end

C_WRITE_A: begin
    if (ready) begin
        cpu_w_en = 1'b1;        // 1-cycle pulse
        cstate_n = C_WRITE_A_WAIT;
    end
end

C_WRITE_A_WAIT: if (write_done) cstate_n = C_READ_A_PRE;

// READ @0x0010
C_READ_A_PRE: begin
    if (ready) begin
        cpu_addr_n = 16'h0010;
        cstate_n = C_READ_A;    // pulse next cycle
    end
end

C_READ_A: begin
    if (ready) begin
        cpu_r_en = 1'b1;        // 1-cycle pulse
        cstate_n = C_READ_A_WAIT;
    end
end

C_READ_A_WAIT: if (read_valid) cstate_n = C_READ_A_CHECK; // go check next cycle
C_READ_A_CHECK: cstate_n = C_RW_PRE;

```

```

// Same-cycle RW @0x0020 (WRITE priority)
C_RW_PRE: begin
    if (ready) begin
        cpu_addr_n = 16'h0020;
        cpu_wdata_n = 16'h1234;
        cstate_n = C_RW_SAME_CYCLE;
    end
end

C_RW_SAME_CYCLE: begin
    if (ready) begin
        cpu_w_en = 1'b1;      // both high same cycle
        cpu_r_en = 1'b1;
        cstate_n = C_RW_WAIT_W;
    end
end

C_RW_WAIT_W:  if (write_done) cstate_n = C_RW_WAIT_R;
C_RW_WAIT_R:  if (read_valid) cstate_n = C_RW_CHECK;  // check next cycle
C_RW_CHECK:   cstate_n = C_DONE;

C_DONE:       cstate_n = C_DONE;

default:      cstate_n = C_RESET;
endcase
end

//Sequential: state, outputs, pipeline, checks
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        cstate    <= C_RESET;
        cpu_addr   <= 16'h0000;
        cpu_wdata  <= 16'h0000;
        expect1    <= 16'hBEEF;
    end
end

```

```

    expect2    <= 16'h1234;
    rdata_q    <= 16'h0000;
    read_valid_q <= 1'b0;
end else begin

    // advance state & outputs

    cstate    <= cstate_n;
    cpu_addr  <= cpu_addr_n;
    cpu_wdata <= cpu_wdata_n;

    // 1-cycle capture of read data/valid

    rdata_q    <= cpu_rdata;
    read_valid_q <= read_valid;

    // ----- Check #1 (one cycle after read_valid) -----
    if (cstate == C_READ_A_CHECK) begin
        if (rdata_q != expect1) begin
            $display("[CPU] ERROR: READ @0x0010 expected %h, got %h @ %0t", expect1, rdata_q, $time);
            $fatal;
        end else begin
            $display("[CPU] PASS : READ @0x0010 matched %h @ %0t", rdata_q, $time);
        end
    end

    //Check #2 (one cycle after read_valid)
    if (cstate == C_RW_CHECK) begin
        if (rdata_q != expect2) begin
            $display("[CPU] ERROR: READ @0x0020 expected %h, got %h @ %0t", expect2, rdata_q, $time);
            $fatal;
        end else begin
            $display("[CPU] PASS : READ @0x0020 matched %h @ %0t", rdata_q, $time);
        end
    end
end

```

```

        end
    end
endmodule

```

7. Complete system TestBecnh code:

● system TestBecnh

```

module tb_top;

    reg clk;    //Clock and Reset
    reg rst_n;

    initial clk = 1'b0;

    always #5 clk = ~clk; //100 MHz

    initial begin
        rst_n = 1'b0;    //async active-low reset
        #25;
        rst_n = 1'b1;
    end

    wire [15:0] cpu_addr;//CPU Controller interface
    wire [15:0] cpu_wdata;
    wire [15:0] cpu_rdata;
    wire    cpu_r_en;
    wire    cpu_w_en;
    wire    ready;
    wire    read_valid;
    wire    write_done;

    wire [15:0] mem_addr;    //Controller Memory interface
    wire [15:0] mem_wdata;
    wire [15:0] mem_rdata;

```

```

wire    mem_r_en;
wire    mem_w_en;

// DUTs
memory_controller u_ctrl (
    .clk    (clk),
    .rst_n  (rst_n),
    //CPU side
    .cpu_addr (cpu_addr),
    .cpu_wdata (cpu_wdata),
    .cpu_rdata (cpu_rdata),
    .cpu_r_en (cpu_r_en),
    .cpu_w_en (cpu_w_en),
    .ready   (ready),
    .read_valid(read_valid),
    .write_done(write_done),

    //memory side
    .mem_addr (mem_addr),
    .mem_wdata (mem_wdata),
    .mem_rdata (mem_rdata),
    .mem_r_en (mem_r_en),
    .mem_w_en (mem_w_en)
);

simple_memory u_mem (
    .clk    (clk),
    .rst_n  (rst_n),
    .mem_addr (mem_addr),
    .mem_wdata(mem_wdata),
    .mem_rdata(mem_rdata),
    .mem_r_en (mem_r_en),
    .mem_w_en (mem_w_en)

```

```

);

simple_cpu u_cpu (
    .clk      (clk),
    .rst_n    (rst_n),
    //to controller
    .cpu_addr (cpu_addr),
    .cpu_wdata (cpu_wdata),
    .cpu_r_en (cpu_r_en),
    .cpu_w_en (cpu_w_en),
    //from controller
    .cpu_rdata (cpu_rdata),
    .ready      (ready),
    .read_valid(read_valid),
    .write_done(write_done)
);

initial begin
    $dumpfile("wave.vcd");
    $dumpvars(0, tb_top);
end

// checks:
//when CPU issues READ+WRITE same cycle with ready=1,
//ensure write_done occurs before read_valid.
//ensure write_done/read_valid are 1-cycle pulses.
reg saw_rw_same_cycle;
reg expect_wd_first;
reg expect_rv_next;

reg prev_write_done;
reg prev_read_valid;

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        saw_rw_same_cycle <= 1'b0;
        expect_wd_first <= 1'b0;
        expect_rv_next <= 1'b0;
        prev_write_done <= 1'b0;
        prev_read_valid <= 1'b0;
    end else begin
        //detect same-cycle RW from CPU when controller is ready
        if (ready && cpu_r_en && cpu_w_en) begin
            saw_rw_same_cycle <= 1'b1;
            expect_wd_first <= 1'b1;
            expect_rv_next <= 1'b0;
        end

        //order check: write_done must come before read_valid
        if (saw_rw_same_cycle) begin
            if (expect_wd_first && read_valid) begin
                $display("[TB] ERROR: read_valid asserted before write_done after same-cycle RW (t=%0t)", $time);
                $fatal;
            end

            if (expect_wd_first && write_done) begin
                expect_wd_first <= 1'b0;
                expect_rv_next <= 1'b1;
            end

            if (expect_rv_next && read_valid) begin
                //success path: saw write_done, then saw read_valid
                expect_rv_next <= 1'b0;
                saw_rw_same_cycle <= 1'b0;
            end
        end
    end
end

```

```

//pulse-width checks: should be 1 cycle each
if (prev_write_done && write_done) begin
    $display("[TB] ERROR: write_done wider than 1 cycle at t=%0t", $time);
    $fatal;
end

if (prev_read_valid && read_valid) begin
    $display("[TB] ERROR: read_valid wider than 1 cycle at t=%0t", $time);
    $fatal;
end

prev_write_done <= write_done;
prev_read_valid <= read_valid;
end
end

//finish sim
initial begin
    #1000;
    $display("[TB] Simulation finished at t=%0t", $time);
    $finish;
end

endmodule

```


8. Complete system simulation screenshots:



Figure 6 Complete system simulation

9. Conclusion

The controller (IDLE/WRITE/READ) with async reset and ready/valid meets the 2-cycle service target and 1-cycle RAM read latency.

Tests confirm WRITE priority on same-cycle READ+WRITE (write_done before read_valid), and both strobes are single-cycle.

Data checks pass (0xBEEF@0x0010, same-cycle RW→0x1234), verified in both unit (tb_mc) and system (tb_top) testbenches.